FELEMN: Toward Efficient Feature-Level Machine Unlearning for Exact Privacy Protection

Zhigang Wang, Yizhen Yu, Mingxin Li, Jian Lou, Ning Wang, Yu Gu, Shen Su, Yuan Liu, Hui Jiang, and Zhihong Tian, *Senior Member, IEEE*

***Abstract*—Data privacy protection legislation around the world has increasingly enforced the "right to be forgotten" regulation, generating a surge in research interest in machine unlearning (MU), which aims to remove the impact of training data from machine learning models upon receiving revocation requests from data owners. There exist two major challenges for the performance of MU: the execution efficiency and the inference interference. The former requires minimizing the computational overhead for each execution of the MU mechanism, while the latter calls for reducing the execution frequency to minimize interference with normal inference services. Nowadays most MU studies focus on the sample-level unlearning setting, leaving the other paramount feature-level setting under-explored. Adapting these existing techniques to the latter turns out to be non-trivial. The only known feature-level work achieves an *approximate* unlearning guarantee, but suffers from degraded model accuracy and still leaves the inference interference challenge unsolved. We are therefore motivated to propose FELEMN, the first FEature-Level Exact Machine uNlearning method that overcomes both of the above-mentioned hurdles. For the MU execution efficiency challenge, we explore the impact of different feature partitioning strategies on the preservation of semantic relationships for maintaining model accuracy and MU efficiency. For the inference interference challenge, we propose two batching mechanisms to combine as many individual unlearning requests to be processed together as possible, while avoiding potential privacy issues coming with falsely postponing unlearning requests, which is grounded on theoretical analysis. Experiments on five real datasets show that our FELEMN outperforms up-to-date competitors with up to 3× speedup for each MU execution, and 50% runtime reduction by mitigating inference interference.**

***Index Terms*—Machine unlearning, privacy protection, execution efficiency, inference interference.**

Received 7 January 2025; revised 12 August 2025; accepted 14 September 2025. Date of publication 23 September 2025; date of current version 10 November 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62572139, in part by the Key-Area Research and Development Program of Guangdong Province under Grant 2023B1111050010, in part by Guangdong S&T Program under Grant 2024B0101010002, in part by the National Natural Science Foundation of China under Grant U2436208 and Grant 62372129, in part by the Project of Guangdong Key Laboratory of Industrial Control System Securityunder Grant 2024B1212020010, and in part by the Key Program of the Key Laboratory of Computing Power Network and Information Security, Ministry of Education under Grant 2024ZD016. Recommended for acceptance by G. Cong. *(Zhigang Wang and Yizhen Yu are co-first authors.) (Corresponding authors: Jian Lou; Ning Wang; Hui Jiang.)*

Zhigang Wang, Ning Wang, Shen Su, Yuan Liu, and Zhihong Tian are with the Cyberspace Institute of Advanced Technology, Guangzhou University, and Guangdong Key Laboratory of Industrial Control System Security, and Huangpu Research School of Guangzhou University, Guangzhou 510006, China (e-mail: wangzhiganglab@gmail.com; wangninggzu@gmail.com; shensu@gzhu.edu.cn; liuyuan@gzhu.edu.cn; tianzhihong@gzhu.edu.cn).

Yizhen Yu and Mingxin Li are with the Faculty of Information Science and Engineering, Ocean University of China, Qingdao 266100, China (e-mail: yuyizhen@stu.ouc.edu.cn; liumingxin@stu.ouc.edu.cn).

Jian Lou is with the School of Software Engineering, Sun Yat-sen University, Zhuhai 519082, China (e-mail: louj5@mail.sysu.edu.cn).

Yu Gu is with the School of Computer Science and Engineering, Northeastern University, Shenyang 110819, China (e-mail: guyu@mail.neu.edu.cn).

Hui Jiang is with China United Network Communications Group Corporation Limited, Beijing 100032, China (e-mail: jiangh272@chinaunicom.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TKDE.2025.3613659, provided by the authors.

Digital Object Identifier 10.1109/TKDE.2025.3613659

I. Introduction

DATASETS crawled from our digital lives have fueled the unprecedented prevalence of machine learning (ML)-based services [1]. A typical workflow of nowadays ML-based services, also known as **Machine Learning as a Service (MLaaS)** [2], roughly consists of two stages. First, the server trains an ML model on the training dataset collected from data owners, and subsequently the trained ML model serves inference requests from end-users. However, the success of ML models is increasingly haunted by privacy concerns, as data owners worry about their privacy leakage from trained ML models under the MLaaS environment. This stems from the inherent memorization characteristic of ML models [3], [4], which inadvertently stores training data in model weights. This consequently incurs severe privacy risks of revealing sensitive information of data owners through inference output [5].

Partly prompted by such privacy concerns, data privacy legislations worldwide, including European Union's GDPR [6] and California's CCPA [7], enforce the **right to be forgotten (RTBF)**. In the context of MLaaS, RTBF stipulates that the server fulfills the obligation to thoroughly remove the training data information from the ML model upon its data owner's request to revoke their previously contributed data [6]. To comply with RTBF, a straightforward approach is to retrain the ML model from scratch on the remaining dataset that excludes the data to be revoked (referred to as Retraining-from-scratch hereafter). Although ensuring thorough data forgotten, it is practically infeasible due to the heavy computational cost of retraining for every data removal request. This spawns an emerging research field—**machine unlearning (MU)** [8], which aims to make ML services compliant with RTBF while maximizing computational savings and minimizing interference with normal inference services.

MU has quickly garnered growing research interest and many promising approaches have been proposed. Existing approaches can be roughly categorized as approximate unlearning and exact unlearning. The former trades a certain degree of deviation from

1041-4347 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:25:11 UTC from IEEE Xplore. Restrictions apply.

the RTBF restriction (and therefore a certain amount of data privacy) for unlearning efficiency. Representative approximate unlearning methods include gradient ascent- [9] and Newton step-based [10] unlearning techniques. In contrast, the latter insists that the model obtained through MU should be exactly the same as the one obtained from retraining-from-scratch in terms of prediction behaviors, aiming for strict compliance with RTBF legislation and avoiding potential privacy issues of approximate unlearning as identified by [11], [12], [13]. **This paper thereby focuses on this exact MU problem**. Exact unlearning methods instead achieve efficiency boosts either by leveraging the special structure of ML models [14], [15], [16], [17], [18], or by redesigning the training paradigm to craft special structures amenable to efficient unlearning for general ML models [19].

The seminal work [8] in exact unlearning proposes the "Sharding-Isolating-Slicing-Aggregating" (SISA) method, which tackles the **MU execution efficiency challenge** by altering the training paradigm as follows: 1) partitioning training samples into several disjoint subsets; 2) training a series of sub-models on each data subset in an incremental way (i.e., slicing by slicing); and 3) finally performing inference by aggregating the outputs of all trained sub-models. This training paradigm greatly facilitates exact unlearning as retraining workloads are now confined only to sub-models involving data removal requests. Following SISA, recent works further enhance MU execution efficiency by designing prominent partitioning policies to confine the impact of a single data removal as much as possible [20], [21], [22], and extend to different types of models [19], [21], [22], [23], [24], [25]. Orthogonal to the MU execution efficiency challenge, a very recent work [26], *Eraser*, identifies another critical **inference interference challenge** that emerges when processing MU requests in the more practical scenario of MLaaS. That is, frequent executions of MU mechanisms, e.g., once per revocation request, can significantly undermine the responsiveness of normal inference services. This calls for selectively batching multiple unlearning requests together to reduce the total number of MU executions. Such a batching strategy should be carefully designed for two reasons. On the one hand, it should batch as many unlearning requests as possible to minimize inference interference. On the other hand, it should execute unlearning immediately if postponing certain unlearning requests can lead to distinct prediction behavior between the otherwise timely unlearned model, ensuring the exactness of unlearning to avoid potential privacy issues.

Research Gap and Challenges: As a nascent research field, most existing machine unlearning studies are limited to the sample-wise unlearning scenario, where data removal is requested and handled in a sample-by-sample way. It remains relatively under-explored for the arguably equally paramount scenario of feature-level unlearning, where data owners wish to revoke partially sensitive features of their data rather than the entire datum. For example, the training data can contain thousands of posts from many customers, each associated with a personal home address. A celebrity, as well as other individuals, might require the removal of the address feature in their posts while leaving other non-sensitive features intact.

Existing coarse-grained, sample-level MU approaches like SISA fail to intelligently accommodate this fine-grained feature-level revocation requirement. By indiscriminately excluding all samples impacted by the revoked feature, these approaches drastically reduce— or even eliminate—usable training data for ML models, resulting in unacceptably poor accuracy performance. Recent works [27], [28] are among the few existing relevant studies. They follow the approximate unlearning techniques to tackle the unlearning efficiency challenge. However, the model accuracy also heavily degrades because they directly reset the value of the revoked feature with random noise to compensate for the unlearning approximation.

For the inference interference challenge, although *Eraser* [26] as the first pioneer is tailored for multiple sample-level revocations, its batching strategy remains applicable to our feature-level scenario. However, when deciding the retraining timing, there exist redundant computations and comparisons. By our tests, it indeed reduces the frequency of performing retraining operations but the substantial runtime penalty ultimately diminishes its practical utility.

Our Work: Motivated by the above, we propose FELEMN: **FE**ature-**L**evel **E**xact **M**achine u**N**learning approach to overcome both MU execution inefficiency and inference interference hurdles for the feature-level unlearning problem. FELEMN adheres to the exact unlearning premise due to its strict compliance with RTBF regulations. It studies dataset partition for better per-unlearning efficiency and batching strategy for minimizing inference interference.

To improve the MU execution efficiency, FELEMN partitions the features of samples into several subsets and replicates the ground-truth label. Then, samples with partial features within a subset can train the corresponding sub-model independently. Once a feature is revoked, we can efficiently perform exact unlearning by only retraining affected sub-models. However, different from existing sample-centric partitioning, this feature-centric partitioning destroys the semantic relationship among features. That impairs the accuracy of each sub-model. We explore different policies to remedy this drawback, including random yet overlapped partitioning and greedy heuristic partitioning. The former relies on the overlapping degree to balance prediction accuracy and unlearning efficiency. The latter extracts the relationship among features as a complete weighted graph and then uses lightweight streaming graph partitioning to obtain subsets. That insulates users from the tedious low-level details, but the unlearning performance is sensitive to the location of the revoked feature in the streaming sequence. Users can make a smart choice to balance their favorites. Besides, we formally prove that our new design can guarantee exact MU.

After settling on the per-execution design for feature-level MU, we move to address the second challenge of inference interference. The intuition is that we can postpone the execution of certain MU operations while maintaining the inference consistency (i.e., achieving the same inference result as if MU were executed per revocation), due to the robustness of ensembling responses from all sub-models. Taking on this intuition, we derive two specific batching mechanisms corresponding to two popular ensemble mechanisms: a hard '0-1' variant and a

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:25:11 UTC from IEEE Xplore. Restrictions apply.

soft variant with a probability distribution. For the former, our in-depth analysis reveals that it is often unnecessary to aggregate all sub-models to maintain the inference consistency, which can be leveraged to actively batch multiple revocations. Our optimized variant, B-FELEMN, can adaptively and efficiently determine the batching mechanism based on aggregated voting scores from clean sub-models only (i.e., the sub-models without revocation requests on their training partition). For the latter, we provide an enhanced design by estimating the similarity between dirty sub-models (with received but unprocessed revocations) and clean sub-models based on the features involved in their training. This enables B-FELEMN to further increase the batch size and reduce the frequency of MU executions, thereby better mitigating inference interferences.

Our main contributions are summarized below.

- We are the first to systematically study both MU execution efficiency and inference interference challenges for exact feature-level machine unlearning.
- We propose FELEMN for unlearning efficiency with a theoretical guarantee in compliance with RTBF, where several fine-grained partitioning policies with different favorites are presented for prominent prediction accuracy and high MU efficiency.
- We propose two batching mechanisms to minimize inference interference, respectively for hard and soft voting. Clean sub-models specified by revoked features are used to safely make correct predictions, so as to delay the retraining of dirty sub-models to share common operations.
- We perform extensive experiments on real datasets against existing MU approaches, including both exact and approximate ones, under single and multiple revocation settings. Our proposed methods yield superior accuracy and efficiency. We also validate the strong unlearning effectiveness of our proposed methods using the SOTA feature inference attack [5].

Organization In the following, Section II summarizes related works. Sections III and IV present our FELEMN and B-FELEMN approaches respectively. Section V reports experiments and Section VI finally concludes this paper.

II. Related Work

Existing MU works can be categorized into *exact* and *approximate unlearning*, based on whether or not the model derived by unlearning approaches is consistent with the one retrained from scratch.

Exact Unlearning: We overview works by their focused ML models. Early pioneers focus on the special models [29], [30] where parameters can be directly derived from the summation of sample-specific values. That simplifies MU because the contribution of revoked samples can be directly subtracted. For the classic tree structure, such as random forests and gradient boosting decision trees (GBDT), researchers construct robust split criteria for each tree node against revocations [15], [16], [17], [18]. For recommendation models, the efficiency is enhanced by utilizing top-k filters over sparse matrices [19]. Besides, Ginart et al. propose to quickly unlearn a k-means clustering model by a divide-and-conquer approach [14].

For deep neural network models, recently the most representative MU framework is SISA [8]. Inspired by ensemble learning, SISA horizontally partitions samples into disjoint sub-datasets, and each of them is used to train a sub-model. It finally aggregates the results from all sub-models during inference. Here a sample is used in a single sub-model. If it is revoked, then only one sub-model, instead of all, needs to be re-trained. Huynh et al. [20] follow SISA but group samples by their ground-truth labels for better efficiency. Besides, for horizontal federated learning, Shaik et al. employ SISA for unlearning and devise an attention approach to adapt to dynamically changing samples [24]. Differently, Tao et al. give a new aggregation algorithm against the impact of a sample in a client [23]. In general, the above methods work well for sample-level revocations. However, when processing our focused feature-level revocations, they degrade to learning from scratch because in most cases nearly all samples with the revoked feature will be affected. In particular, the *Eraser* framework [26] pays attention to remedying the inference interference when multiple revocations continuously arrive. However, it is tailored for only hard voting in the aggregation phase of SISA, and even so, it still has many redundant operations. By contrast, our solution goes further and can work well for both hard and soft voting.

Approximate Unlearning: The optimization goal of exact unlearning is to confine the time-consuming retraining workloads, but cannot eliminate them. Some works [31], [32], [33] try to completely remove such behaviors, but with an approximate unlearning, which requires the unlearned model parameters to be close to the ones derived by re-training from scratch. Their main idea of these works is to estimate how much the revoked samples have contributed to the model parameters, by analyzing the first-order or second-order gradients. Then they can rollback the model by subtracting the estimation from the already released version. Clearly, the accuracy of estimation plays an important role and hence becomes a hot research topic. Several works [27], [31], [32], [34], [35], [36] thereby focus on certified unlearning, which associates the estimation with differential privacy [37]. It can provide a statistical guarantee that an unlearned model parameter cannot be distinguished from the one derived by retraining from scratch. However, most of them still focus on the sample-level unlearning tasks. Besides, some researchers state that the approximate scheme still has serious security flaws [11], [12], [13].

As we know, there exists only one work targeting at our feature-level focus. It approximates the impact of revoked features and then unlearn the contribution from the revoked feature [27]. However, such an approximation incur accuracy penalty as validated in our experiments. Besides, dropping a client in vertical federated learning resembles feature-level MU. However, the related solutions either simply subtract estimated contributions [38] or ignore the relationship among clients [39], both of which also yield accuracy penalty.

Other Related Works: Beyond centralized MU, federated unlearning (FU) has emerged as an active research area. Liu et al. [40] develop an approximation method that modifies the loss function to enable efficient retraining on revocation data. Qiu et al. [41] adapt the SISA framework [8] to federated

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:25:11 UTC from IEEE Xplore. Restrictions apply.

settings by clustering clients into shards using K-means and training dedicated sub-models to confine unlearning workloads. Tao et al. [23] propose a version-control approach where model parameters are checkpointed upon processing new training data, allowing rollback to pre-revocation states when needed. These sample-level techniques cannot work for feature-level MU.

In contrast, Zhang et al. [42] propose a gradient transformation approach that converts unlearning requests into direct parameter updates. While effective for graph-structured data by leveraging topological relationships, this method is fundamentally incompatible with non-graph models.

We are aware that there exist some feature-specific MU works in computer vision [43], [44], [45], where "features" denote spatial pixel regions in an image matrix rather than dimensional attributes in a vector in our context. These methods differ fundamentally in their technical approach. In fact, they primarily employ loss function modifications to eliminate contributions from revoked features, representing approximate rather than exact unlearning solutions. Note that UIB [45] further eliminates the patterns and biases embedded in revoked data. Besides, Shaik propose to adaptively unlearn multimodality data based on Reinforcement Learning [24].

MU represents a passive privacy-preserving mechanism that is initiated by data owners upon request. In contrast, active protection techniques—such as homomorphic encryption, secure multi-party computation, and differential privacy [46], [47], [48], [49], proactively prevent privacy leakage during utilization of sensitive data. These complementary approaches (passive MU and active protections) can be integrated to enhance the security of MLaaS platforms.

Last but not least, an emerging research direction concerns feature-level privacy attacks. Luo et al. [5] develop methods to reconstruct original feature values, while Jayaraman et al. [50] investigate sensitive attribute inference attacks. These newly identified vulnerabilities underscore the critical need for robust feature-level privacy protection mechanisms.

III. FELEMN: Feature-Level Exact Machine Unlearning

Our MU architecture FELEMN re-designs the conventional training algorithm to confine retraining workloads. It consists of three components, the feature-centric training algorithm, the voting-based inference rule, and the exact unlearning with theoretical guarantee. Below, we first formalize the exact feature-level unlearning problem and then derive FELEMN.

Problem Formalization: Let $T : D \rightarrow \mathcal{H}$ denote the training algorithm T that maps a dataset D to a model in the hypothesis space $\mathcal{H}$, where D consists of n samples and each sample is with d features from $\mathcal{A} = \{A_1, A_2, \ldots, A_d\}$. Let $R : T(D) \times D \times \mathcal{A} \rightarrow \mathcal{H}$ be an unlearning algorithm tailored for T to make the originally published model $T(D)$ unlearned for a feature $A_i \in \mathcal{A}$. Given a revoked feature A_i, a **Feature-level Exact Machine Unlearning** requires a model output denoted by R is equivalent to the one retrained by T on $D_{\mathcal{A}}A_i$, i.e., the probability $\Pr[T(D_{\mathcal{A}}A_i)] = \Pr[R(T(D), D, A_i)]$. Here $D_{\mathcal{A}}A_i$ is the remaining dataset after removing elements of the revoked feature A_i.

Algorithm 1: Training the Ensemble Model.

Input: Dataset D, feature set $\mathcal{A}$, # of sub-models K
Output: Ensemble model $\mathcal{M}$
Partitioning features in $\mathcal{A}$ into K subsets $\mathcal{A}_1, \ldots, \mathcal{A}_K$;
for each subset $\mathcal{A}_i$ **do**
  Projecting D as D_i based on $\mathcal{A}_i$;
  Training the sub-model M_i using D_i.
end for
return $\mathcal{M} = \{M_1, M_2, \ldots, M_K\}$

Training Ensemble Model with Partitioned Features: An ensemble model $\mathcal{M}$ consists of multiple sub-models M_i trained on different sub-datasets [8]. When data revocation requests affect some sub-datasets, only the related sub-models need retraining, which confines the retraining workloads. Following this design, in FELEMN, a sub-dataset D_i consists of samples with only partial features. Specifically, as shown in Algorithm III, we construct K feature combinations $\{\mathcal{A}_1, \mathcal{A}_2, \ldots, \mathcal{A}_K\}$ based on some partitioning strategy discussed later, each of which involves a subset of features in $\mathcal{A}$. Then, D_i is extracted from the original training data D by projecting it to the features in $\mathcal{A}_i$. Subsequently, we can train the sub-model M_i based on D_i and the total K sub-models compose the ensemble model used in the downstream inference tasks.

However, unlike sample-centric MU, where samples are naturally more independent of each other, partitioning features in FELEMN potentially destroys the implicit semantic relationships across features. Meanwhile, the partitioning should be size-balanced. Otherwise, the sub-model with an overly large subset will face heavy training and unlearning workloads. Additionally, it is more likely to be affected by a revocation. Considering both the accuracy and efficiency aspects of MU, we investigate three partitioning policies.

1) *Random. Overlapped Random Partitioning:* Each of the K feature subsets independently and randomly samples c features from $\mathcal{A}$. A feature might appear in more than one subset, so as to compensate for accuracy loss. Usually, $c \geq \frac{|\mathcal{A}|}{K}$. The partitioning procedure can be quickly executed. However, a feature revocation can affect several subsets due to its multiple appearances. That slows down the MU processing. Users can specify c to balance accuracy and efficiency.

2) *GreedyMax. Greedy Partitioning with Maximal Dependency:* We expect to group features with high dependency into the same subset for prominent accuracy while guaranteeing the equal-sized balance constraint for MU efficiency. Towards this end, we build a complete weighted graph G with features as vertices and mutual information MI(A_i, A_j) [51] as the weight for an edge between features A_i and A_j. Then feature partitioning is transformed into graph partitioning. We particularly employ existing lightweight disjoint streaming partitioning [52] to solve this problem where features are sequentially streamed. For each newly arrived feature, it is greedily assigned to the subset with the maximal weight summation (contributed by

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:25:11 UTC from IEEE Xplore. Restrictions apply.

its already assigned neighbors), s.t. the balance constraint that $c = \frac{|\mathcal{A}|}{K}$.

3) *GreedyMin. Greedy Partitioning with Minimal Dependency:* This framework is similar with *GreedyMax*. The difference is that the criterion of "maximal weight summation" is changed into "minimal weight summation". Then features with less significant dependent features are grouped. The idea behind that is to avoid over-fitting for each sub-model, and then improve the final inference accuracy.

Voting-based Inference: FELEMN employs the widely-used voting strategy to deliver aggregated inference results. Given an inference request $\mathbf{x}$, it collects the individual inferences from K sub-models and then makes the final prediction based on a majority-voting mechanism. Specifically, FELEMN supports both hard voting and soft voting strategies. Let $\mathbb{Y}$ denote the set of ground-truth labels. The soft inference result of sub-model $M_i(\mathbf{x})$ is a vector of size $|\mathbb{Y}|$ with the probability distribution of all labels in $\mathbb{Y}$. For hard voting, $M_i(\mathbf{x})$ is a one-hot vector where the bit w.r.t. the most likely label is 1 and others are 0. Eq. (1) mathematically shows the final predicted label based on the majority voting rule.

$$\mathcal{M}(\mathbf{x}) = \arg\max_{y \in \mathbb{Y}} \left(\sum_{i=1}^{K} M_i(\mathbf{x})(y) \right) \tag{1}$$

Exact Two-phase Unlearning: Since the training algorithm (Algorithm III) contains two phases, partitioning and training, the unlearning processing correspondingly has two procedures. Below we present the detailed design and prove that it satisfies the exact unlearning definition. In particular, we group our partitioning policies into two categories, random and greedy, and give different solutions respectively.

1) *Random Replacement:* For the random-based partitioning, we traverse each subset $\mathcal{A}_i$ to check whether or not the revoked feature A_r belongs to it. If yes, another feature A_x is randomly selected from $(\mathcal{A} - \mathcal{A}_i)$ to replace A_r. We then confine retraining to sub-models M_i with updated subsets $\mathcal{A}'_i$.

2) *Semi-Restreaming:* For greedy max/min partitioning, the partitioning output is sensitive to the streaming order. We first identify the location of A_r in the streaming sequence. Assume that it is the t-th streamed feature. Then features streamed before t are kept unchanged, which can be directly copied from $\mathcal{A}_i$ to $\mathcal{A}'_i$ for $i \in [1, K]$. While other features need to be re-streamed, because their original assignments depend on A_r but now A_r is removed. Afterwards, we retrain each sub-model M_i if $\mathcal{A}_i \neq \mathcal{A}'_i$ happens. Because partial features keep their partitioning locations unchanged while others are re-streamed, we call this procedure as *Semi-Restreaming*.

Exact MU Guarantee: Theorems 1 and 2 respectively show that our two designs can satisfy the exact MU definition.

Theorem 1: The probability of the model $\mathcal{M}'$ under random replacement is equal to that acquired by learning from scratch.

Proof: Please see Section I in the supplementary file.

Theorem 2: The probability of the model $\mathcal{M}'$ under semi-restreaming is equal to that acquired by learning from scratch.

Proof: Please see Section II in the supplementary file.

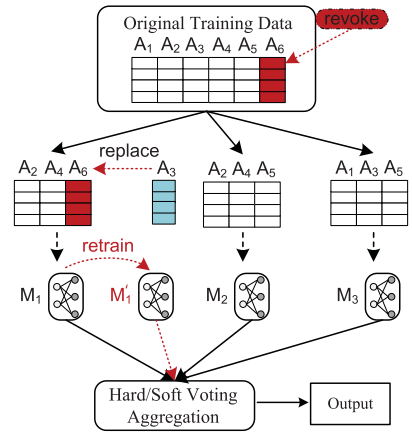


Fig. 1. The framework of feature-level machine unlearning FELEMN.

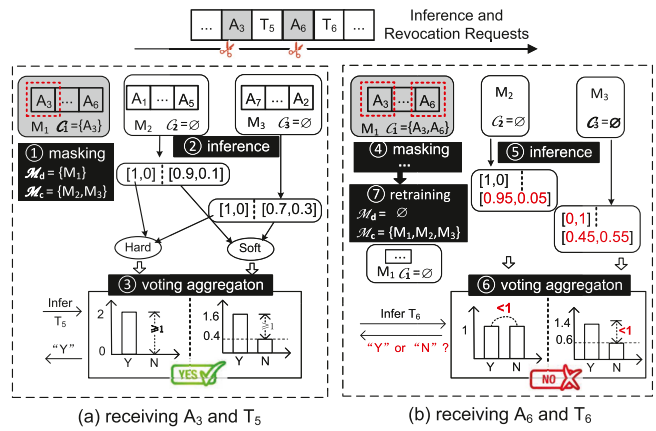


Fig. 2. The workflow of B-FELEMN—a batched variant of FELEMN.

A Concrete Example: Fig. 1 shows an example where features $\mathcal{A} = \{A_1, \ldots, A_6\}$ are grouped into three subsets $\mathcal{A}_1 = \{A_2, A_4, A_6\}$, $\mathcal{A}_2 = \{A_2, A_4, A_5\}$ and $\mathcal{A}_3 = \{A_1, A_3, A_5\}$, using the Overlapped Random Partitioning policy. Sub-models M_1, M_2 and M_3 are then respectively trained and constitute the ensemble model $\mathcal{M}$, to perform aggregation inference. Once A_6 is revoked, $\mathcal{A}_1$ is affected. The new subset $\mathcal{A}'_1$ replaces A_6 with another randomly selected feature A_3 to guarantee the exact MU. The corresponding M_1 is retrained as M'_1 to participate in answering the subsequent inference tasks. By contrast, M_2 and M_3 are kept unchanged for efficiency.

IV. B-FELEMN: Batching Optimization of FELEMN for Multiple Revocations

This section presents an optimized variant of FELEMN to alleviate the inference interference.

A. Problem Analysis About Inference Interference

MLaaS servers need to cope with continuously received revocation requests [6], [7]. That possibly leads to repeated retraining of a certain sub-model. For example, at the top bar of Fig. 2, revocations A_3 and A_6 both belong to $\mathcal{A}_1$ but arrive at different

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:25:11 UTC from IEEE Xplore. Restrictions apply.

times. Then M_1 is retrained twice. Because the inference service pauses during retraining, the frequent retraining behaviors clearly yield heavy inference interference.

On the other hand, the internal sub-models operate as non-transparent systems, remaining invisible to end -users. Consequently, the Right to Be Forgotten (RTBF) right required by privacy legislations can be strictly guaranteed—provided that the inference result remains independent of any sub-models trained using the revoked feature. This condition is achievable under a majority voting mechanism, since the final aggregated predictions do not necessarily rely on all constituent sub-models. As shown in Fig. 2(a), for the inference request T_5, M_2 and M_3 respectively output [1, 0] and [1, 0] under hard voting, or [0.9, 0.1] and [0.7, 0.3] under soft voting. During voting aggregation, if the difference between the highest and second-highest aggregated values exceeds 1, we can confidently determine the correct answer—irrespective of the output from model M_1. This observation demonstrates that masking model M_1 allows us to delay its retraining in response to the revocation of feature A_3. Consequently, this strategy creates an opportunity to combine the retraining requirements for the currently arrived A_3 and the upcoming A_6, enabling a single retraining update to mitigate the inference interference.

B. A Basic Batching Framework

Motivated by the analytical insights from Section IV-A, we propose B-FELEMN, a batched variant of FELEMN designed to enhance inference throughput when handling multiple feature revocations. The fundamental distinction lies in their retraining strategies. FELEMN performs immediate retraining of affected sub-models upon receiving revocation requests, while B-FELEMN strategically delays retraining until the sub-model plays the pivotal role in majority voting.

Overview Design: B-FELEMN employs a batched revocation mechanism to consolidate multiple revocation requests for single-round retraining. Upon receiving a revocation request, the system stores it in a pre-allocated cache $\mathcal{C}_i$ associated with its corresponding sub-model M_i. This design categorizes sub-models into two distinct groups: Clean sub-models ($\mathcal{M}_c$) containing no pending revocations, i.e., $\mathcal{C}_i = \emptyset,\ \forall M_i \in \mathcal{M}_c$, and Dirty sub-models ($\mathcal{M}_d$), each of which contains received but unprocessed revocations, i.e., $\mathcal{C}_i \neq \emptyset,\ \forall M_i \in \mathcal{M}_d$.

As outlined in Algorithm 2, B-FELEMN executes the following procedure during inference.

- *Initial aggregation:* For an input inference request $\mathbf{x}$, all clean sub-models in $\mathcal{M}_c$ are executed, with their outputs aggregated into a prediction vector $R \in \mathbb{R}^{|\mathbb{Y}|}$ (Line 1).
- *Confidence evaluation:* The system compares the margin between the highest and second-highest values in R against the cardinality of $\mathcal{M}_d$ to measure the confidence of answering $\mathbf{x}$ solely by clean sub-models in $\mathcal{M}_c$, as shown in Inequation (2).

$$(\mathrm{Max}(R) - \mathrm{SecMax}(R)) \geq |\mathcal{M}_d| \tag{2}$$

If this condition holds, we can guarantee that the dirty sub-models cannot flip the final prediction, even in the worst-case scenario where all dirty sub-models vote for the second-highest value in R. This ensures the clean sub-models' output is deterministically correct.

- *Dynamic retraining:* When confidence margins are insufficient, a dirty sub-model $M_r \in \mathcal{M}_d$ is selected for batched retraining where all cached revocations in $\mathcal{C}_r$ are processed, followed by its participation in the voting aggregation (Lines 3-6).

Algorithm 2: Inference in B-FELEMN.

Input: Inference $\mathbf{x}$, caches of revocations $\mathcal{C}_1$—$\mathcal{C}_K$, and sets of sub-models $\mathcal{M}_d$&$\mathcal{M}_c$
Output: The predicted output of $\mathbf{x}$
1: $R \leftarrow \sum_{M_i \in \mathcal{M}_c} M_i(\mathbf{x})$
2: **while** Inequation (2) cannot hold & $\mathcal{M}_d \neq \emptyset$ **do**
3: Retraining a sub-model $M_r \in \mathcal{M}_d$ by invoking FELEMN
4: $\mathcal{C}_r \leftarrow \emptyset$
5: Removing the retained M_r from $\mathcal{M}_d$ to $\mathcal{M}_c$
6: $R \leftarrow R + M_r(\mathbf{x})$
7: **end while**
8: **return** $\arg\max_{y \in \mathbb{Y}}(R(y))$

A Concrete Example: Fig. 2 demonstrates how B-FELEMN works in a binary classification task. Consider an MLaaS server processing an alternating sequence of revocation requests (A_3, A_6) and inference jobs (T_5, T_6). As depicted in sub-figure(a), upon receiving A_3, B-FELEMN delays retraining of sub-model M_1. It stores A_3 into the cache $\mathcal{C}_1$ and then masks M_1 by moving it from $\mathcal{M}_c$ to $\mathcal{M}_d$ (①). This masking prevents M_1 from participating in subsequent inferences. Thus, when processing the upcoming T_5, only M_2 and M_3 in $\mathcal{M}_c$ are involved in inference (②). Nevertheless, during voting aggregation(③) their outputs satisfy Inequation (2) for both hard and soft voting branches. This confidence evaluation result allows B-FELEMN to output the label "Y", even though M_1 is absent. Differently, sub-figure(b) demonstrates the dynamic retraining. For another revocation A_6, B-FELEMN similarly buffers it into $\mathcal{C}_1$ (④). However, when processing inference request T_6, B-FELEMN fails to satisfy the confidence criterion based solely on the outputs from M_2 and M_3 (⑤&⑥). Now M_1 plays the pivot role in the voting ensemble. B-FELEMN initiates retraining of M_1 so that it can help to correctly predict the output label (⑦). Upon completion of retraining, B-FELEMN clears $\mathcal{C}_1$, thereby restoring M_1 to a validated state and enabling its participation in subsequent inference tasks.

Exact MU Guarantee: Theorem 3 tells us the ensemble model generated by Algorithm 2 satisfies exact MU.

Theorem 3: The probability of the unlearned model under B-FELEMN is equal to that acquired by learning from scratch.

Proof: Please see Section III in the supplementary file.

C. An Enhanced Framework for Soft Voting

Finally, we analyze the negative impact of strict confidence evaluation in Inequation (2) for soft voting. Building upon the

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:25:11 UTC from IEEE Xplore. Restrictions apply.

TABLE I
CORRELATION ANALYSIS

Corr/Datasets	*Adult*	*Diabetes*	*Purchase*	*Covtype*	*Cropland*
$M_j^{\max}$	0.91	0.83	0.94	0.92	0.99
$M_j^{\min}$	0.86	0.77	0.93	0.62	0.98

TABLE II
DETAILS OF DATASETS

Dataset	*# of Features*	*# of Samples*	*# of Classes*
Adult [53]	162	32,561	2
Diabetes [54]	483	101,770	3
Purchase [55]	600	311,519	2
Covtype [56]	144	581,012	7
Cropland [57]	175	325,834	7

observed similarity in inference behaviors across sub-models, we subsequently propose an enhanced variant designed to substantially extend retraining operations.

The Impact of Strict Confidence Evaluation in Soft Voting: In B-FELEMN, we analyze the retraining trigger timing under the worst-case scenario where all dirty sub-models unanimously vote for y'—the class with the second-highest score in R. Specifically, each dirty sub-model $M_i(\mathbf{x}) \in \mathcal{M}_d$ outputs a one-hot vector [0,...,1,...,0], with 1 assigned exclusively to y'. While this strict assumption holds for hard voting where outputs are binary, it does not reflect the reality of soft voting where y' typically receives a probability value significantly lower than 1. Consequently, this overestimation results in excessive false-positive retraining triggers, undermining the efficiency gains of batched updates.

Similarity Inference Behaviors across Sub-models: Recall that each sub-model M_i is trained on a distinct feature subset $\mathcal{A}_i$. When $\mathcal{A}_i$ and $\mathcal{A}_j$ exhibit high feature similarity, M_i and M_j are likely to produce identical inference results. In such cases, we can leverage a clean sub-model M_i to approximate the output of a dirty M_j. Thus, even without retraining M_j, the approximated output can improve the likelihood of satisfying the confidence evaluation criteria. This allows M_j's retraining to be further delayed, accumulating more revocations to improve batch processing efficiency.

We empirically validate this similarity hypothesis using real-world datasets (see Table II). Through our Overlapped Random Partitioning method, we construct 10 distinct feature subsets. For evaluation, we randomly designate one subset $\mathcal{A}_i$ as the target and systematically compute its correlation score $\text{Corr}(\mathcal{A}_i, \mathcal{A}_j)$ with every other subset $\mathcal{A}_j$ $(i \neq j)$ using (3), which leverages mutual information $\text{MI}(A_i, A_j)$ [51] as its foundation. We select the subsets with maximum and minimum scores $\mathcal{A}_j^{\max}$ and $\mathcal{A}_j^{\min}$, then perform 1,000 inferences to measure prediction consistency between M_i and the two selected extreme cases. Table I presents the comparative percentage analysis. The substantial gap observed confirms the prevalence of feature correlations, while demonstrating the effectiveness of (3) in quantifying these relationships. Note that the datasets *Purchase* and *Cropland* exhibit only marginal differences, but both $M_j^{\max}$ and $M_j^{\min}$ have high scores. This condition still permits reliable estimation.

Algorithm 3: Inference in B-FELEMN$_\Delta$.

Input: Inference $\mathbf{x}$, and sets of sub-models $\mathcal{M}_d$&$\mathcal{M}_c$
Output: The predicted output of $\mathbf{x}$

```
R ← Σ_{M_i∈M_c} M_i(x)
R_Δ ← 0
M_Δ ← M_d
cnt_d ← |M_d|
while Inequation (4) cannot hold
    if M_Δ ≠ ∅ then
        Removing a sub-model M_tar from M_Δ
        M_sim ← {M_i ∈ M_c | Corr(M_i, M_tar) > θ}
        if M_sim ≠ ∅ then
            R_Δ ← R_Δ + Σ_{M_i∈M_sim} M_i(x) / |M_sim|
            cnt_d ← cnt_d − 1
        end if
    else
        Retraining a sub-model M_j ∈ M_d by invoking
        FELEMN
        Moving M_j from M_d to M_c
        Re-initializing R, R_Δ, M_Δ and cnt_d
    end if
end while
return arg max_{y∈Y} (R(y))
```

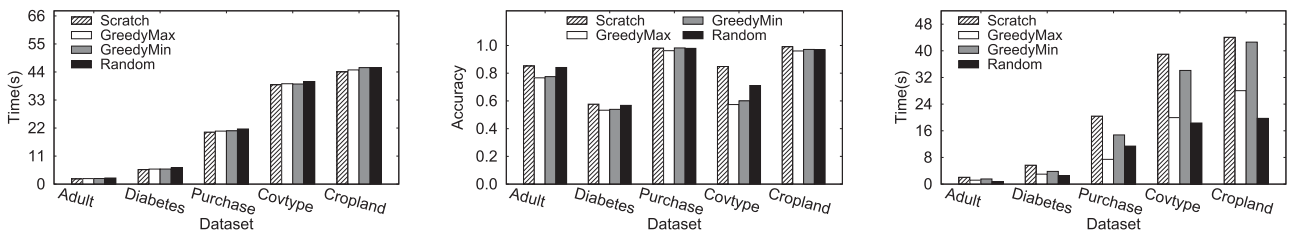
$$\text{Corr}(\mathcal{A}_i, \mathcal{A}_j) = \sum_{A_i \in \mathcal{A}_i} \sum_{A_j \in \mathcal{A}_j} \text{MI}(A_i, A_j) \tag{3}$$

B-FELEMN$_\Delta$ Enhanced by Similarity Estimation: We now present the design of B-FELEMN$_\Delta$, where the estimated outputs from dirty sub-models are utilized for confidence evaluation. Accordingly, Inequation (2) used in B-FELEMN is now reformulated as Inequation (4), where R_Δ represents the estimation information and cnt_d is the number of dirty sub-models failing to perform valid similarity estimation. The core idea is that if we cannot guarantee the correctness of the output solely derived by clean sub-models, we remove a dirty sub-model M_{tar} from $\mathcal{M}_\Delta$ where $\mathcal{M}_\Delta$ is initialized by $\mathcal{M}_d$ and $cnt_d = |\mathcal{M}_d|$. Subsequently, we identify similar clean sub-models M_{sim} corresponding to M_{tar} using (3) with threshold θ. Generally, during the estimation process, cnt_d progressively decreases while R_Δ accumulates increasing information. This gradual improvement enhances the probability of satisfying Inequation (4), ultimately leading to successful confidence evaluation. B-FELEMN$_\Delta$ thereby can delay the retraining operation. It should be noted that for a given M_{tar}, there may exist no clean sub-models satisfying $Corr > \theta$. In this case, we skip M_{tar} while maintaining cnt_d unchanged.

$$(\text{Max}(R + R_\Delta) - \text{SecMax}(R + R_\Delta)) \geq cnt_d \tag{4}$$

Algorithm 3 outlines the inference workflow of B-FELEMN$_\Delta$. Different from B-FELEMN, B-FELEMN$_\Delta$ eliminates the buffering revocation cache mechanism. Upon receiving a revocation, it promptly updates the subset $\mathcal{A}_i$ enabling immediate use of the revised $\mathcal{A}_i$ to identify clean sub-models

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:25:11 UTC from IEEE Xplore. Restrictions apply.



(a) Runtime of ML training (b) Accuracy before unlearning (c) Runtime of unlearning

Fig. 3. Overall performance comparison of feature partitioning policies for FELEMN ($K = 8$, a single revocation).

similar to M_i when required. Specifically, when Inequation (4) cannot hold, B-FELEMN$_\triangle$ attempts to estimate the output of dirty sub-models (Lines 6-12) to augment confidence evaluation. Because of the threshold θ, Inequation (4) may remain unsatisfied even after full estimation of dirty sub-models. In this case, B-FELEMN$_\triangle$ degrades to B-FELEMN. It retrains a dirty sub-model and then re-initializes variables for another round of similarity estimation (Lines 13-17).

Remark: Estimating the scores of dirty sub-modes incurs additional errors, which degrades inference accuracy. However, this does not contradict the exact MU definition, as B-FELEMN$_\triangle$ adopts the same batch design as that in B-FELEMN.

V. Experiments

Experiments are run on a server with an NVIDIA GeForce RTX 3080 GPU (8704 CUDA Cores, 10 GB HBM2), and one Intel Core i9-10900 K CPU (10 Cores, 64 GB DDR4 RAM). *Compared Solutions:* Since our FELEMN is the first work addressing exact feature-level MU tasks, we develop two competitors for comparison. The first approach, termed *Scratch*, removes all samples containing revoked features and then retrains the ML model from scratch using only the remaining data. The second approach, *F-SISA*, extends the SISA framework [8] by training sub-models on sample-level disjoint subsets. Upon detecting a revoked feature, it identifies the affected subsets and retrains only the corresponding sub-models. For comprehensive comparison, we additionally evaluate the approximate feature-level approach *F-Approx* [27] using its default parameter settings. This method estimates the influence of revoked features and subsequently updates the model parameters to achieve unlearning. Besides, for multiple feature revocations, we compare our B-FELEMN and the enhanced B-FELEMN$_\triangle$ with another two baselines. The first is to run FELEMN One-By-One without any delay, termed as *OBO*. The other is *Eraser* [26], that uses the inference results from all sub-models to adaptively delay the retraining timing. Notably *Eraser* only works for hard voting.

Datasets and Benchmarks: We use 5 real datasets in our experiments, as summarized in Table II. They are with different numbers of samples and features, and have different semantic relationships among features, that can validate the applicability of our proposals. We use Multilayer Perceptron with two hidden layers to perform classification tasks, each layer containing 128 neurons. Notably the input layer dynamically adjusts the number of neurons based on how many features are fed. Similarly, the number of neurons in the output layer is equal to the number of class labels.

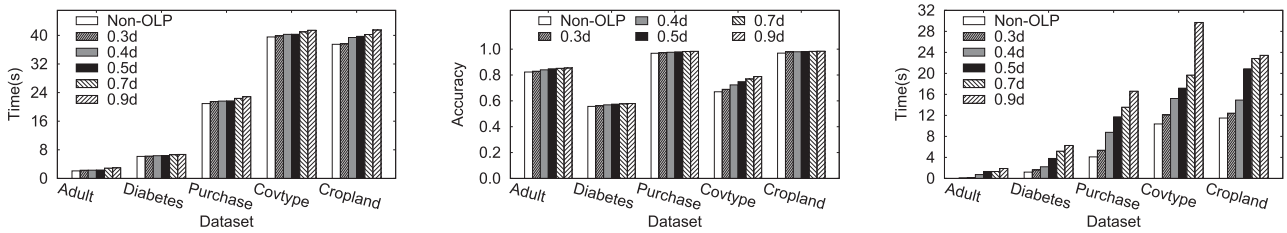
Experimental Designs: Our proposed methods form a cohesive framework with two specialized variants: (1) B-FELEMN for hard-voting mechanisms, and (2) B-FELEMN$_\triangle$ for soft-voting mechanisms. Both implementations build upon the foundational FELEMN approach for retraining models (as shown in Algorithms 2 and 3). To systematically evaluate their contributions, we conduct distinct experimental validations. FELEMN is tested in single-feature revocation scenarios to isolate the benefits of our feature partitioning strategy (Section V-A). B-FELEMN/B-FELEMN$_\triangle$ are evaluated under multi-feature revocation to demonstrate the efficiency of batching optimization (Section V-B). Besides, we report the quantified information leakage about the revoked feature, by adopting the up-to-date feature inference attack [5] against our FELEMN (Section V-C). We finally discuss the technical advancement and generality of our proposals (Section V-D). By default we randomly select the revoked feature and then remove all involved samples. We report the runtime and accuracy metrics using their averages of five tests. For a complete comparison, we also explore performance when only partial samples revoke their values on the selected feature (Section V-A). Notably we run the benchmark with batch size as 128 and learning rate as 0.001. The training process consists of 10 epochs.

A. Evaluation on Unlearning a Single Feature

We report the performance when revoking a single feature. We first test the performance of different partitioning policies in our FELEMN and then compare it against competitors.

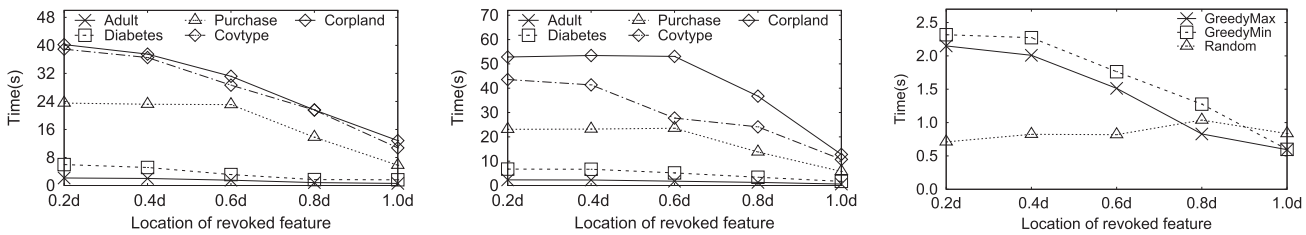
We have three partitioning policies, *Random*, *GreedyMax* and *GreedyMin* (see Section III). Fig. 3 reports their performance in terms of normal training runtime, prediction accuracy of the trained model and unlearning runtime. Here $c = 0.4d$ for *Random*, which is greater than that of greedy heuristics $\frac{d}{K}$. As shown in sub-figure(b), all partitioning policies exhibit the similar accuracy performance with that of the traditional model *Scratch*. Among our policies, *Random* has the best accuracy in most cases because the redundant overlapped features can well preserve the semantic relationship. *Random* also beats greedy-based heuristics on many datasets in unlearning runtime as shown in sub-figure(c), mainly due to its fast replacement operations on limited sub-sets. Sub-figure(a) shows the training runtime comparison. We find that although *Random* has a

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:25:11 UTC from IEEE Xplore. Restrictions apply.



(a) Runtime of ML training (b) Accuracy before unlearning (c) Runtime of unlearning

Fig. 4. Impact of the overlapping degree for the *Random* partitioning policy in FELEMN ($K = 4$, a single revocation).



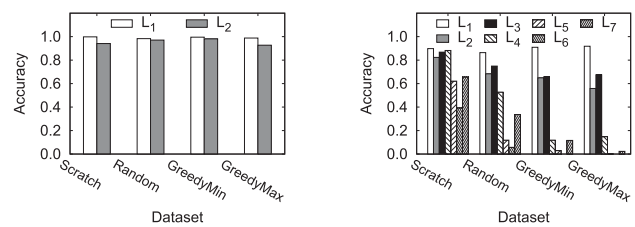
(a) *GreedyMax* (b) *GreedyMin* (c) Overall comparison on *Adult*

Fig. 5. Impact of the revocation location on unlearning runtime, for the greedy partitioning policies *GreedyMax/Min* in FELEMN ($K = 4$, a single revocation).

large sub-set size, the runtime increase is less significant. This is because the number of features just affects the workloads in the input layer which occupy a small fraction in the total training workloads. On the other hand, greedy heuristics usually have poor unlearning efficiency for a random revocation. The reason is that their semi-restreaming partitioning in MU might change many subsets and hence enforce a lot of sub-models to be retrained. In particular, *GreedyMin* has prominent accuracy, while *GreedyMax* makes a good compromise between accuracy and unlearning efficiency. In *Purchase*, they even beat *Random* respectively in accuracy and unlearning runtime.

Both *Random* and greedy heuristics have their specific hyper-parameters. Now we explore their impacts. In Fig. 4 we test *Random* by varying c from $0.3d$ to $0.9d$, indicating the increased overlapping degree. The non-overlapping solution termed as "Non-OLP" is also involved as a baseline. As expected, we observe a linear increase for unlearning runtime in sub-figure(c) since sub-models are retrained in high probability. While, the accompanied benefit is that the semantic relationship can be well preserved. The model accuracy thereby increases from 0.67 to 0.78 at most (*Covtype* in sub-figure(b)). Besides, as explained in Fig. 3(a), there exists a slight increase for the training runtime in sub-figure(a).

We next focus on greedy heuristics. Revoking a feature disturbs the partitioning order during semi-restreaming, that generates non-deterministic impact on unlearning runtime. The performance on *Purchase* in Fig. 3(c) has implicitly revealed this phenomenon. Now Fig. 5 gives detailed analysis by varying the location of the revoked feature A_i in the streaming sequence of size d, ranging from $0.2d$ to $1.0d$ (A_i is the last streamed feature in partitioning). With the increase of revocation location, we observe gradual decrease in unlearning runtime for both greedy



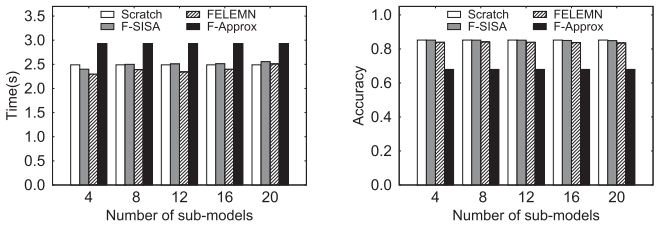
(a) Balanced *Purchase* (b) Imbalanced *Covtype*

Fig. 6. Impact of the class distribution on prediction accuracy before unlearning, under different feature partitioning policies ($K = 4$, FELEMN).

heuristics on all datasets. We achieve the best performance at $1.0d$ where only one sub-set is affected and then one sub-model is retrained. Greedy heuristics even outperform *Random* after $0.8d$, as shown in sub-figure(c).

The classification datasets typically contain multiple class labels with diverse sample distributions. This diversity might affect the accuracy under our partitioning design. Without loss of generality, we use *Purchase* (balanced) and *Covtype* (imbalanced) as representative examples. We group samples by labels and then investigate the fine-grained, label-level accuracy performance. The *Purchase* dataset has two class labels R_1 and R_2. Both of them are well trained due to the roughly balanced distribution of samples. Consequently, as shown in Fig. 6(a), all our partitioning approaches achieve high accuracy on both labels, performing comparably to the non-partitioned baseline *Scratch*. In contrast, the *Covtype* dataset contains 7 imbalanced labels where sample distribution is highly skewed—ranging from 48.8% (L_1) to 0.5% (L_7). Due to the extreme data sparsity,

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:25:11 UTC from IEEE Xplore. Restrictions apply.



(a) Runtime of ML training (b) Accuracy after unlearning

Fig. 7. Overall performance comparison of competing unlearning solutions, with K varying from 4 to 10 (*Adult*, a single revocation).

minority-class samples (L_4–L_7) cannot be effectively memorized during training, leading to sub-optimal accuracy, as evidenced by the *Scratch* baseline in Fig. 6(b). Feature partitioning further exacerbates this issue by introducing sparsity into each sub-model, resulting in an overall accuracy drop (as validated in Fig. 3(b)). However, *Random* partitioning exhibits less degradation than greedy-based methods, as its overlapping mechanism partially mitigates sparsity. Interestingly, greedy partitioning yields a sharp accuracy dichotomy: while majority-class samples (L_1–L_3) achieve high accuracy (even surpassing *Scratch* by 2% for L_1), minority-class performance deteriorates significantly. This stems from the partitioning process being dominated by majority-class samples, which retains well-preserved feature semantics while suppressing noise from minority classes.

In general, *Random* has better prediction accuracy. Its unlearning runtime is sensitive to its built-in hyper-parameter c, but robust to the revocation location. End-users can specify c to balance accuracy and efficiency on demand. While greedy heuristics avoids this choice, its unlearning runtime is dominated by revocation location in real-world. We can carefully tune the streaming order by analyzing the revocation frequency of different features, to enhance the performance. This will be investigated as future work. Since *Random* has steady unlearning performance, in the following experiments, we use it with $0.4d$ as the default partitioning policy.

We finally perform an overall comparison. We first report the runtime and the model accuracy when normally training the original ML models under different solutions. As shown in Fig. 7(a), the runtime of *Scratch* is independent of K since it does not train sub-models. For *F-SISA*, samples are evenly partitioned onto K sub-sets. A large K setting means more sub-models need to be trained, that is time-consuming. However, the increase of K also decreases the number of samples in each sub-set and hence the training time of the related sub-model. Thus, the total training runtime is comparable to that of *Scratch*. Our FELEMN inherits this advantage by horizontally partitioning samples across sub-models. But it further performs vertical partitioning, i.e., each sub-model only utilizes partial features of a sample. Since each sub-model is trained with fewer features, we achieve roughly 8% runtime improvement. Meanwhile, training with fewer features inevitably generates negative impact on accuracy. However, the degradation is less than 2%, due to the overlapping design. For the approximation approach *F-Approx*, its prediction accuracy heavily degrades (up to 57% on *Covtype* and 32% on average). This is because Gaussian noise is added to the loss function for theoretical guarantee. Besides, its training runtime significantly increases because the modified loss function heavily complicates the computation of gradients.

Fig. 8 explores the unlearning performance under different K settings. We assume that all samples ask to revoke their data on the selected feature, termed as *full revocation*. This is a default setting in all of experiments. As expected, *F-Approx* has the best unlearning runtime but at expense of prediction accuracy and training runtime (as depicted in Fig. 7). *F-SISA* has the similar performance with *Scratch*, regardless of K settings. This is reasonable since all samples have the revoked feature and then any subset as well as sub-model will be affected. Our FELEMN achieves an overall success. It runs roughly 3 times faster than *F-SISA*. One observation is that when K varies, its runtime exhibits fluctuations. This is because the randomly revoked feature might appear in any number of random feature combinations. The number of affected sub-models that need to be retrained (retraining worklodas) dynamically changes.

Different from the full revocation assumption in Figs. 8, 9 focuses on the scenario where partial samples revoke data, namely *partial revocation*. Compared with Fig. 8(a), in Fig. 9(a), both *F-SISA* and FELEMN show the similar performance, since these 50% samples are uniformly distributed across almost all sub-sets. Differently, *F-Approx* runs faster because fewer samples need to calculate the gradient difference. The runtime impact is significant between Figs. 8(d) and 9(b), due to the very small number of affected samples. The runtime of FELEMN decreases from 23.5 s down to 1.9 s ($K = 20$). For *F-Approx*, it varies from 5.5 s to 0.1 s.

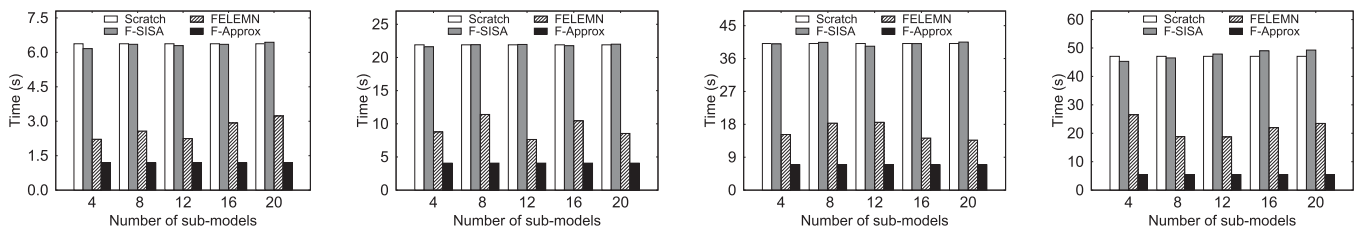
B. Evaluation on Unlearning Multiple Features

In this suit of experiments, we evaluate the performance of our batching design when unlearning multiple features. We select 500 samples as inference tasks from the test dataset and 10 features as revocation tasks from $\mathcal{A}$. We submit them to simulate the request sequence where revocation tasks are evenly distributed among inference tasks. Below, we analyze the performance of our basic B-FELEMN and optimized B-FELEMN$_\triangle$, respectively under hard voting and soft voting[1].

Under the hard voting rule, we compare B-FELEMN with the one-by-one implementation of FELEMN (*OBO*) and the up-to-date batching framework *Eraser*. For better understanding the difference between *Eraser* and B-FELEMN, we separate the unlearning process into two procedures, including preprocessing of determining the retraining timing & computing the final inference result (such as Lines 1-2 in Algorithm 2 for our B-FELEMN) and physical retraining (such as Line 3 in Algorithm 2 in B-FELEMN). Figs. 10 and 11 respectively report the runtime comparison when varying K from 4 to 20.

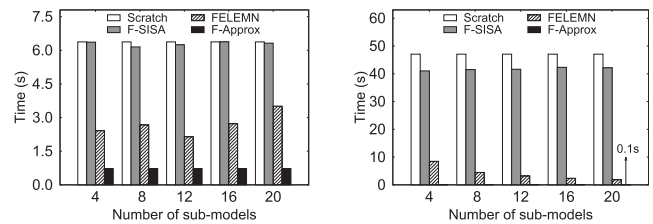
[1]We omit some sub-figures in Figs. 10, 11, and 12 for brevity, because they exhibit the similar performance trend. Please see Figs. 1–3 for full experiment results in Section IV in the supplementary file.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:25:11 UTC from IEEE Xplore. Restrictions apply.



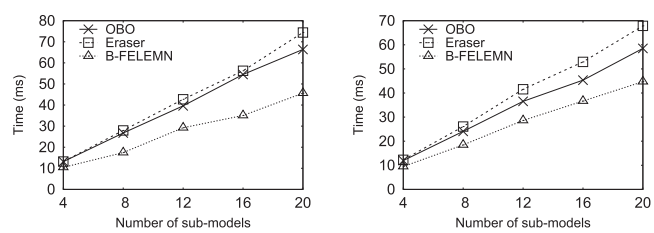
(a) *Diabetes* (b) *Purchase* (c) *Covtype* (d) *Cropland*

Fig. 8. Comparison on unlearning runtime of competing solutions with K varying from 4 to 20. All samples eliminate the selected revoked feature.



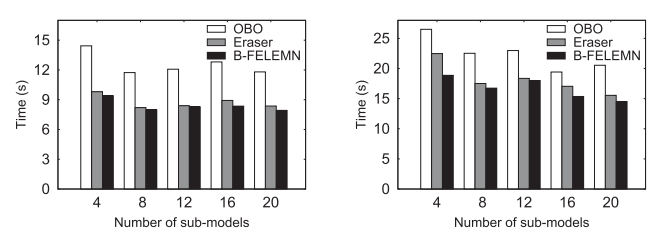
(a) *Diabetes* (b) *Cropland*

Fig. 9. Comparison on unlearning runtime of competing solutions with K varying from 4 to 20. $\chi\%$ of samples eliminate the selected revoked feature. $\chi = 50$ for *Diabetes* and 1 for *Cropland*.



(a) *Purchase* (b) *Covtype*

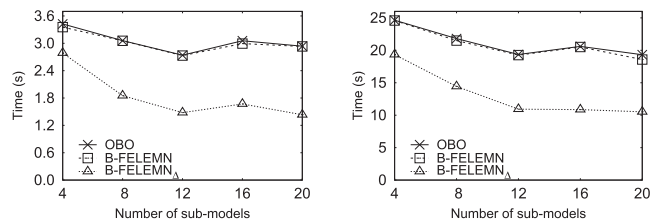
Fig. 10. Comparison on preprocessing & inference runtime (hard voting, multiple revocations).



(a) *Purchase* (b) *Covtype*

Fig. 11. Comparison on retraining runtime (hard voting, multiple revocations).

Fig. 10 shows the runtime of all approaches linearly increases, because the number of sub-models running inference is proportional to K. Here Among them, our B-FELEMN performs the best, since dirty sub-models affected by revocations are excluded from inference. By contrast, *Eraser* has the worst performance. It runs inference on both clean and dirty sub-models. Worse,



(a) *Diabetes* (b) *Covtype*

Fig. 12. Comparison on unlearning runtime (soft voting, multiple revocations).

dirty sub-models need to be run again once they are physically retrained, that incurs redundant inference operations. As a result, *Eraser* yields up to 60% runtime degradation compared with B-FELEMN (*Purchase*, $K = 20$), and 15% even compared with *OBO* (*Covtype*, $K = 20$).

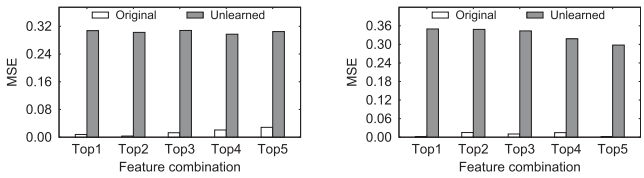
As explained in Figs. 8, 11 shows that all competing approaches have slight variation on retraining runtime with K. Nevertheless, our B-FELEMN always has the best performance. Compared with *Eraser*, it decreases the runtime by 15% at most (*Covtype*, $K = 4$). This is because *Eraser* enforces all dirty sub-models to be retrained, if aggregated voting scores from existing sub-models cannot help to make correct prediction. Differently, our design gradually retrains dirty sub-models. After retraining a sub-model, it timely checks the judging condition again (Line 13 in Algorithm 2). It thereby might terminate retraining in advance to enhance the batching benefit. Compared with *OBO* without any delaying optimization, the speedup is up to 35% (*Purchase*, $K = 4$).

We next test the soft voting. Because *Eraser* tailored for hard voting is excluded, here we do not separate the preprocessing&inference runtime and the retraining runtime. Instead, we directly report the total unlearning runtime. Our B-FELEMN and optimized B-FELEMN$_\triangle$ are run in comparison with *OBO*. Fig. 12 reports the overall runtime comparison. It is observed that the B-FELEMN slightly outperforms *OBO*. This phenomenon empirically validates our analysis in Section IV-C. That is, it cannot work as expected when inference results from clean sub-models with soft voting and dirty sub-models with hard voting estimation are simply aggregated. That heavily limits the batching benefit. Differently, B-FELEMN$_\triangle$ outperforms the above two methods by a large margin, with up to 50% improvement (*Diabetes*, $K = 20$). This significant advantage is achieved by estimating the inference results of dirty sub-models.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:25:11 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPARISON ON ACCURACY AFTER UNLEARNING FOR *OBO*, B-FELEMN (*BF* FOR SHORT) AND B-FELEMN$_\triangle$ ($BF_\triangle$ FOR SHORT)

Dataset	*Adult*			*Diabetes*			*Purchase*			*Covtype*			*Cropland*		
Methods	*OBO*	*BF*	$BF_\triangle$	*OBO*	*BF*	$BF_\triangle$	*OBO*	*BF*	$BF_\triangle$	*OBO*	*BF*	$BF_\triangle$	*OBO*	*BF*	$BF_\triangle$
K=4	84.48	84.52	84.18	56.40	56.42	56.37	97.92	97.86	97.54	74.55	74.37	74.07	97.96	98.02	98.02
K=8	84.30	84.46	84.28	57.54	57.48	57.34	98.46	98.38	97.98	72.56	72.52	72.14	98.00	98.02	98.00
K=16	83.46	83.50	83.24	57.70	57.65	57.35	98.10	98.04	97.68	71.17	71.17	71.15	97.34	97.36	97.44



(a) *Combinations from MIF* (b) *Combinations from LIF*

Fig. 13. Unlearning power of FELEMN (*Diabetes*).

This design effectively delays the retraining process and then more revocations can be batched for high throughput. Besides, we generally find that the runtime gap between B-FELEMN$_\triangle$ and the other two increases with K. Our explanation is that when more sub-models are available, the probability of successfully finding correlated clean sub-models as estimation becomes higher.

Although B-FELEMN$_\triangle$ achieves prominent unlearning efficiency, approximating dirty sub-models with clean sub-models inevitably incurs errors. We thereby report the accuracy comparison of different approaches in Table III. We observe that on all datasets, *OBO* and B-FELEMN have the similar performance. By contrast, B-FELEMN$_\triangle$ generates a very slight degradation with less than 0.6%. That validates the effectiveness of our correlation matching optimization.

C. Evaluation on Unlearning Power of FELEMN

We finally validate the unlearning power of FELEMN. Recently Luo et al. have designed an ML-based feature inference attack technique [5]. It attempts to recover the attack target's feature values by utilizing the already trained model's parameters and non-target features. Luo et al. use the mean square error (MSE) to measure the recovery accuracy. Accordingly, we quantify the unlearning power of FELEMN by the difference of MSE values when attacking the Original model and the Unlearned model. We particularly use the feature attribution toolkit SHAP [58] to find the Most/Least 5 Important Features for model training, and divide them into two candidate revocation sets, termed as *MIF* and *LIF* respectively. In each set, we enumerate all possible revocation combinations and report the cases with top-5 MSE differences.

Fig. 13 plots the results on *Diabetes*. As validated in Ref. [5], the attack performance on the Original model is prominent, i.e., MSE ≈ 0. However, we see a consistent and significant degradation on the Unlearned model. For example, in the top-1 case in Fig. 13(b), the MSE value increases from 0.0015 up to 0.3498, yielding a gap of 0.35 (the average gap is 0.23 for all combination cases). The large gap indicates the increased difficulty of recovery. That is, FELEMN can effectively unlearn the impact of target feature's values.

D. Analysis of Technical Advancement and Generality

Through rigorous technical exposition and experimental validation, we conduct a comprehensive comparative analysis to establish the advancements of our method over existing representative solutions and thoroughly investigate its generalization capabilities in practical deployment contexts.

Technical Advancement: On one hand, existing exact sample-level MU methods exhibit unacceptable accuracy degradation when processing feature-level revocations. Across all real-world datasets examined, only 5% of samples in *Adult* and 11% in *Diabetes* miss partial features, and 0% in others. This implies that revoking any single feature renders nearly the complete dataset unusable for training, leading to substantial accuracy penalty. For the approximate feature-level MU work *F-Approx* [27], our experiments in Fig. 7 also validates its disadvantage in accuracy, stemming from its zero-setting operation and the random noise perturbation. On the other hand, when handling multiple revocations, *Eraser* requires executing all sub-models in $\mathcal{M}_c \cup \mathcal{M}_d$ to aggregate inference results for consistency analysis, to decide whether or not launching retraining. This process not only maintains the same inefficiency as standard inference but further deteriorates runtime performance due to the exhaustive comparison between the highest-confidence class label and all other candidates (see Fig. 10). In contrast, our B-FELEMN demonstrates that the final prediction depends solely on the top two highest-confidence class labels. This insight enables substantial optimization, elimination of redundant comparisons, and selective execution of only relevant sub-models in $\mathcal{M}_c$, collectively achieving significant computational efficiency gains.

Generality for Federated Learning: While our current experiments are conducted in a centralized testbed with single-GPU configuration, our solution can also work in decentralized environments with distributed GPU resources, particularly suited for Federated Learning (FL) frameworks where a central server coordinates multiple clients for collaborative ML training. Similar to sample-level MU in FL, the implementation necessitates a modification of the conventional training paradigm. Clients first partition features into several subsets like $\mathcal{A}_i$ and then collaboratively train the sub-model w.r.t. $\mathcal{A}_i$ for $i \in [1, K]$. We have investigated the deployment in an industrial setting at China Unicom Limited, to train models using federated data

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:25:11 UTC from IEEE Xplore. Restrictions apply.

sourced from several provincial branches. The critical challenge lies in achieving consistent feature space partitioning across all participating clients. Otherwise, a single feature revocation might affect more than one sub-model. A possible solution is to designate a single client to perform the initial partitioning operation and subsequently distribute the resulting configuration to all other clients. However, this uniform partitioning scheme may underperform with non-IID (non-Independent and Identically Distributed) data in FL. As demonstrated in Fig. 3, our three partitioning policies exhibit varying accuracy and unlearning runtime across different datasets (serving as FL clients). The generally optimal *Random* partitioning policy further requires dataset-specific tuning of its overlapping degree hyper-parameter (see Fig. 4). These findings suggest that uniform partitioning configurations may compromise either accuracy or efficiency for FL clients. We will address this limitation in future work.

VI. Conclusion

This paper investigates the feature-level revocation problem. The proposed FELEMN approach employs the ensemble learning framework to confine the impact a feature has on the model training, in order to efficiently perform unlearning operations. We particularly study several feature partitioning policies with different favorites, to balance prediction accuracy and unlearning efficiency. Besides, when processing multiple feature revocations, we propose to batch MU operations to remedy the inference interference. Extensive experiments validate the effectiveness and efficiency of our proposals.

References

[1] M. S. Bartlett, G. Littlewort, M. G. Frank, C. Lainscsek, I. R. Fasel, and J. R. Movellan, "Recognizing facial expression: Machine learning and application to spontaneous behavior," in *Proc. 2005 IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, 2005, vol. 2, pp. 568–573.

[2] Amazon SageMaker, "Sagemaker-machine learning at scale," 2018. [Online]. Available: https://aws.amazon.com/sagemaker

[3] M. Fredrikson, S. Jha, and T. Ristenpart, "Model inversion attacks that exploit confidence information and basic countermeasures," in *Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur.*, 2015, pp. 1322–1333.

[4] N. Carlini, C. Liu, Ú. Erlingsson, J. Kos, and D. Song, "The secret sharer: Evaluating and testing unintended memorization in neural networks," in *Proc. USENIX Secur.*, 2019, pp. 267–284.

[5] X. Luo, Y. Wu, X. Xiao, and B. C. Ooi, "Feature inference attack on model predictions in vertical federated learning," in *Proc. IEEE 37th Int. Conf. Data Eng.*, 2021, pp. 181–192.

[6] A. Mantelero, "The EU proposal for a general data protection regulation and the roots of the 'right to be forgotten," ' *Comput. Law Secur. Rev.*, vol. 29, no. 3, pp. 229–235, 2013.

[7] "California consumer privacy act (CCPA)," State of California Office of the Attorney General, 2024. [Online]. Available: https://oag.ca.gov/privacy/ccpa

[8] L. Bourtoule et al., "Machine unlearning," in *Proc. 42nd IEEE Symp. Secur. Privacy*, 2021, pp. 141–159.

[9] Y. Liu et al., "Backdoor defense with machine unlearning," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 280–289.

[10] Z. Cao, J. Wang, S. Si, Z. Huang, and J. Xiao, "Machine unlearning method based on projection residual," in *Proc. IEEE 9th Int. Conf. Data Sci. Adv. Analytics*, 2022, pp. 1–8.

[11] N. G. Marchant, B. I. P. Rubinstein, and S. Alfeld, "Hard to forget: Poisoning attacks on certified machine unlearning," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 7691–7700.

[12] H. Hu et al., "A duty to forget, a right to be assured? exposing vulnerabilities in machine unlearning services," in *Proc. 31st Annu. Netw. Distrib. Syst. Secur. Symp.*, 2023, pp. 1–17.

[13] A. Thudi, H. Jia, I. Shumailov, and N. Papernot, "On the necessity of auditable algorithmic definitions for machine unlearning," in *Proc. 31st USENIX Secur. Symp. USENIX Assoc.*, 2022, pp. 4007–4022.

[14] A. Ginart, M. Y. Guan, G. Valiant, and J. Zou, "Making AI forget you: Data deletion in machine learning," in *Proc. 33rd Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 3518–3531.

[15] H. Lin, J. W. Chung, Y. Lao, and W. Zhao, "Machine unlearning in gradient boosting decision trees," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2023, pp. 1374–1383.

[16] Z. Wu, J. Zhu, Q. Li, and B. He, "DeltaBoost: Gradient boosting decision trees with efficient machine unlearning," in *Proc. ACM Manage. Data*, 2023, vol. 1, no. 2, pp. 1–26 .

[17] J. Brophy and D. Lowd, "Machine unlearning for random forests," in *Proc. 38th Int. Conf. Mach. Learn.*, 2021, pp. 1092–1104.

[18] S. Schelter, S. Grafberger, and T. Dunning, "HedgeCut: Maintaining randomised trees for low-latency machine unlearning," in *Proc. 2021 Int. Conf. Manage. Data*, 2021, pp. 1545–1557.

[19] S. Schelter, S. Grafberger, and M. D. Rijke, "Snapcase - regain control over your predictions with low-latency machine unlearning," *Proc. VLDB Endowment*, vol. 17, no. 12, pp. 4273–4276, 2024.

[20] N. V. Huynh, D. T. Hoang, D. N. Nguyen, and E. Dutkiewicz, "Joint coding and scheduling optimization for distributed learning over wireless edge networks," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 2, pp. 484–498, Feb. 2022.

[21] M. Chen, Z. Zhang, T. Wang, M. Backes, M. Humbert, and A. Zhang, "Graph unlearning," in *Proc. 2022 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 499–513.

[22] C. Chen, F. Sun, M. Zhang, and B. Ding, "Recommendation unlearning," in *Proc. ACM Web Conf.*, 2022, pp. 2768–2777.

[23] Y. Tao, C. Wang, M. Pan, D. Yu, X. Cheng, and D. Wang, "Communication efficient and provable federated unlearning," *Proc. VLDB Endowment*, vol. 17, no. 5, pp. 1119–1131, 2024.

[24] T. Shaik et al., "FRAMU: Attention-based machine unlearning using federated reinforcement learning," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 10, pp. 5153–5167, Oct. 2024.

[25] M. Kurmanji, E. Triantafillou, and P. Triantafillou, "Machine unlearning in learned databases: An experimental analysis," *Proc. ACM Manage. Data*, vol. 2, no. 1, 2024, Art. no. 49.

[26] Y. Hu, J. Lou, J. Liu, F. Lin, Z. Qin, and K. Ren, "ERASER: Machine unlearning in Mlaas via an inference serving-aware approach," in *Proc. 2024 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2024, pp. 3883–3897.

[27] A. Warnecke, L. Pirch, C. Wressnegger, and K. Rieck, "Machine unlearning of features and labels," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2023, pp. 1–18.

[28] L. Han, N. Luo, H. Huang, J. Chen, and M.-A. Hartley, "Towards independence criterion in machine unlearning of features and labels," 2024, *arXiv:2403.08124*.

[29] Y. Cao and J. Yang, "Towards making systems forget with machine unlearning," in *Proc. IEEE Symp. Secur. Privacy*, 2015, pp. 463–480.

[30] S. Schelter, "'Amnesia'- machine learning models that can forget user data very fast," in *Proc. Conf. Innov. Data Syst. Res.*, 2020, pp. 1–9.

[31] S. Neel, A. Roth, and S. Sharifi-Malvajerdi, "Descent-to-Delete: Gradient-based methods for machine unlearning," in *Proc. 32nd Int. Conf. Algorithmic Learn. Theory*, 2021, pp. 931–962.

[32] A. Golatkar, A. Achille, and S. Soatto, "Eternal sunshine of the spotless net: Selective forgetting in deep networks," in *Proc. IEEE/CVF Comput. Vis. Found.*, 2020, pp. 9301–9309.

[33] A. Peste, D. Alistarh, and C. H. Lampert, "SSSE: Efficiently erasing samples from trained machine learning models," 2021, *arXiv:2107.03860*.

[34] C. Guo, T. Goldstein, A. Y. Hannun, and L. V. D. Maaten, "Certified data removal from machine learning models," in *Proc. 37th Int. Conf. Mach. Learn.*, 2020, pp. 3832–3842.

[35] E. Chien, C. Pan, and O. Milenkovic, "Certified graph unlearning," in *Proc. New Front. Graph Learn. Workshop*, 2022, pp. 1–32.

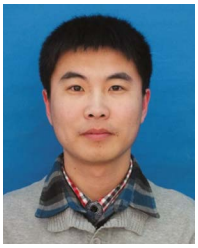
[36] V. Gupta, C. Jung, S. Neel, A. Roth, S. Sharifi-Malvajerdi, and C. Waites, "Adaptive machine unlearning," in *Proc. 35th Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 16319–16330.

[37] C. Dwork, K. Kenthapadi, F. McSherry, I. Mironov, and M. Naor, "Our data, ourselves: Privacy via distributed noise generation," in *Proc. Adv. Cryptology – EUROCRYPT*, 2006, pp. 486–503.

[38] Z. Deng et al., "Vertical federated unlearning on the logistic regression model," *Electronics*, vol. 12, no. 14, 2023, Art. no. 3182.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:25:11 UTC from IEEE Xplore. Restrictions apply.

[39] Z. Wang, X. Gao, C. Wang, P. Cheng, and J. Chen, "Efficient vertical federated unlearning via fast retraining," *ACM Trans. Internet Techn.*, vol. 24, no. 2, 2024, Art. no. 11.
[40] Y. Liu, L. Xu, X. Yuan, C. Wang, and B. Li, "The right to be forgotten in federated learning: An efficient realization with rapid retraining," in *Proc. IEEE INFOCOM 2022 Conf. Comput. Commun.*, 2022, pp. 1749–1758.
[41] H. Qiu, Y. Wang, Y. Xu, L. Cui, and Z. Shen, "FedCIO: Efficient exact federated unlearning with clustering, isolation, and one-shot aggregation," in *Proc. IEEE Int. Conf. Big Data*, 2023, pp. 5559–5568.
[42] H. Zhang, B. Wu, X. Yang, X. Yuan, X. Liu, and X. Yi, "Dynamic graph unlearning: A general and efficient post-processing method via gradient transformation," in *Procs. ACM Web Conf. 2025*, 2025, pp. 931–944.
[43] H. Gu, W. Ong, C. S. Chan, and L. Fan, "Ferrari: Federated feature unlearning via optimizing feature sensitivity," in *Proc. 38th Int. Conf. Neural Inf. Process. Syst.*, 2024, pp. 24150–24180.
[44] S. Moon, S. Cho, and D. Kim, "Feature unlearning for pre-trained GANs and VAEs," in *Proc. 38th AAAI Conf. Artif. Intell.*, 2024, pp. 21420–21428.
[45] L. Han, H. Huang, D. Scheinost, M. Hartley, and M. R. Martıńez, "Unlearning information bottleneck: Machine unlearning of systematic patterns and biases," 2024, *arXiv:2405.14020*.
[46] H. Fereidooni et al., "SAFELearn: Secure aggregation for private federated learning," in *Proc. IEEE Secur. Privacy Workshops*, 2021, pp. 56–62.
[47] C. Zhang, S. Li, J. Xia, W. Wang, F. Yan, and Y. Liu, "BatchCrypt: Efficient homomorphic encryption for cross-silo federated learning," in *Proc. USENIX Annu. Techn. Conf.*, 2020, pp. 493–506.
[48] M. Abadi et al., "Deep learning with differential privacy," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 308–318.
[49] Z. Bu, Y.-X. Wang, S. Zha, and G. Karypis, "Differentially private optimization on large model at small cost," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 3192–3218.
[50] B. Jayaraman and D. Evans, "Are attribute inference attacks just imputation?," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 1569–1582.
[51] J. Zhang, G. Cormode, C. M. Procopiuc, D. Srivastava, and X. Xiao, "PrivBayes: Private data release via Bayesian networks," *ACM Trans. Database Syst.*, vol. 42, no. 4, 2017, Art. no. 25.
[52] I. Stanton and G. Kliot, "Streaming graph partitioning for large distributed graphs," in *Proc. 18th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2012, pp. 1222–1230.
[53] UCI Mach. Learn. Repository, "Adult," 1996. [Online]. Available: https://archive.ics.uci.edu/dataset/2/adult
[54] UCI Mach. Learn. Repository, "Diabetes 130-US hospitals for years 1999-2008," 2014. [Online]. Available: https://archive.ics.uci.edu/dataset/296/diabetes+130-us+hospitals+for+years+1999-2008
[55] Kaggle, "Acquire valued shoppers challenge," 2014. [Online]. Available: https://www.kaggle.com/competitions/acquire-valued-shoppers-challenge/data
[56] UCI Mach. Learn. Repository, "Covertype," 1998. [Online]. Available: https://archive.ics.uci.edu/dataset/31/covertype
[57] UCI Mach. Learn. Repository, "Crop mapping using fused optical-radar data set," 2020. [Online]. Available: https://archive.ics.uci.edu/dataset/525/crop+mapping+using+fused+optical+radar+data+set
[58] S. M. Lundberg and S. Lee, "A unified approach to interpreting model predictions," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 4768–4777.



Zhigang Wang received the PhD degree in computer software and theory from Northeastern University, China, in 2018. He is currently an associate professor with Guangzhou University. His research interests include distributed graph processing and machine learning. He was the recipient of the CCF Outstanding Doctoral Dissertation Award (2018), ACM QINGDAO Rising Star Award (2019), and the second prize of CCF Natural Science Award (2023).He is a member of IEEE and the China Computer Federation (CCF).

Yizhen Yu is currently working toward the bachelor's degree in software engineering with the Ocean University of China. His research interests include machine unlearning and deep learning. He was awarded the first prize in the 2024 National Undergraduate Mathematics Competition in Shandong Province and the second prize in the 2023 National Software and Information Technology Talent Competition in Shandong Province.

Mingxin Li received the bachelor's degree from Tianjin Polytechnic University in 2022. She is currently working toward the master's degree in the School of Computer Science and Technology with the Ocean University of China. Her research focuses on machine unlearning.

Jian Lou received the BS degree from Zhejiang University and the PhD degree from Hong Kong Baptist University. He was an associate professor with Xidian University. He was a postdoc hosted by prof. Li Xiong with the Department of Computer Science, Emory University. He is currently an associate professor with Sun Yat-sen University. His research interests include trustworthy machine learning and privacy-preserving machine learning.

Ning Wang received the PhD degree in computer software and theory from Northeastern University, China, in 2017. She was a visiting PhD student in Nanyang Technological University during 2014 to 2015. She is currently an associate professor with Guangzhou University. Her research interests include Big Data process, graph analysis, and differential privacy protection. She was the recipient of the ACM QINGDAO Rising Star Award (2021).

Yu Gu received the PhD degree in computer software and theory from Northeastern University, China, in 2010. He is currently a professor and the PhD supervisor with Northeastern University, China. His current research interests include Big Data analysis, distributed computation, and graph data management. He was the recipient of the second prize of CCF Natural Science Award in 2023. He is a senior member of CCF.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:25:11 UTC from IEEE Xplore. Restrictions apply.

Shen Su received the BE, ME, and PhD degrees from the Harbin Institute of Technology. He is currently a professor with the Cyberspace Institute of Advanced Technology, Guangzhou University, China. He has authored or coauthored more than 60 journals and conference papers with more than 1200 citations and seven ESI high-indexed papers. His research interests include blockchain security, DNS, route modelling, route security, cyber range, vehicular networks, and wire less sensor networks.

Yuan Liu received the BSc degree from the Harbin Institute of Technology, Harbin, China, in 2010, and the PhD degree from the School of Computer Engineering, Nanyang Technological University (NTU), Singapore, in 2014. She is currently a professor with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou, China. Her research interests include trust-based incentive mechanism design, federated learning, trust management, blockchain consensus protocols, and blockchain powered artificial intelligence.

Hui Jiang received the BS degree from the Department of Internet of Things, Nanjing University of Posts and Telecommunications, Nanjing, China, in 2017, and the PhD in computer science from the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China. She is currently postdoctoral with the China United Network Communications Group Corporation Limited. Her current research interests include federated learning, edge intelligence, and trustworthy distributed AI.

Zhihong Tian (Senior Member, IEEE) received the PhD degree in computer science and technology from the Harbin Institute of Technology. He is currently a professor, and the dean, with the Cyberspace Institute of Advanced Technology, Guangzhou University. He is also a part-time professor with Carlton University. He has authored more than 200 journal and conference papers in computer networks and cyberspace security. He is a senior member of the IEEE Computer Society and CCF.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:25:11 UTC from IEEE Xplore. Restrictions apply.